МИНОБРНАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Техническое задание по курсовому проекту по дисциплине «Функциональное программирование»

Студент группы 5130201/40002 _____ Резчик Д.О.

Преподаватель _____ Моторин Д.Е.

«____» _____ 2026 г.

Санкт-Петербург
2026

Содержание

1 Техническое задание **3**
1.1 Основания для выполнения работы . . . 3
1.2 Исполнитель и соисполнители работы . . . 3
1.3 Цели, задачи и исходные данные . . . 3
1.3.1 Цель работы . . . 3
1.3.2 Задачи . . . 3
1.4 Исходные данные . . . 4
1.5 Основное содержание работы . . . 4
1.5.1 Разработка проекта . . . 4
1.6 Основные требования к выполнению работы . . . 6
1.6.1 Назначение программного комплекса . . . 6
1.6.2 Состав разрабатываемого программного комплекса . . . 6
1.6.3 Технические характеристики . . . 7
1.6.4 Функциональные требования . . . 7
1.6.5 Требования к используемым средствам . . . 7
1.7 Этапы и сроки выполнения работы . . . 7
1.7.1 Диаграмма Ганта . . . 8
1.8 Результаты работы . . . 9

1 Техническое задание

1.1 Основания для выполнения работы

Основанием для выполнения работы является учебный план направления 02.03.01 «Математика и компьютерные науки» Высшей школы технологий искусственного интеллекта Института компьютерных наук и кибербезопасности Санкт-Петербуржского Политехнического университета имени Петра Великого, который предусматривает выполнение курсовой работы по дисциплине «Функциональное программирование».

1.2 Исполнитель и соисполнители работы

Исполнитель: студент учебной группы 5130201/40101 Резчик Денис Олегович.

1.3 Цели, задачи и исходные данные

1.3.1 Цель работы

Целью работы является разработка принципиальной программной имитации роторной шифровальной машины HX-63(Crypto AG, 1963). **Важное ограничение:** итоговый результат не является полным повторением настоящей шифровальной машины HX-63. В связи с недостатком данных и описания работы шифровальной машины, в некоторых местах приняты допущения, достаточные для корректной работы.

1.3.2 Задачи

Для достижения поставленной цели будет необходимо решить следующие задачи:

1. Реализовать основные типы данных (`Rotor`, `MachineState`, `InternalKey`, `MessageKey` и др.) и инварианты этих типов.
2. Реализовать чистые функции построения подстановки сигнала с учётом механизма `re-entry` (`buildSubstitution`).
3. Реализовать шифрование и дешифрование одного символа (`encryptChar`, `decryptChar`), при этом дешифрование должно выполняться через построение обратной подстановки.
4. Реализовать детерминированную функцию шага роторов (step).
5. еализовать шифрование и дешифрование целых текстов (encrypt, decrypt).
6. Реализовать генерацию, сохранение, загрузку и валидацию ключей.
7. Реализовать минимальный интерфейс командной строки для работы с моделью.
8. Покрыть критическую функциональность тестами.

9. Зафиксировать в документации все принятые допущения относительно устройства реальной машины HX-63.

1.4 Исходные данные

Исходными данными в данной работе будут являться ключи, генерирующиеся программой, и тексты, нужные для шифровки-дешифровки.

Виды ключей:

1. Внутренние ключи (порядок роторов, проводки, штифты, модификаторы).
2. Ключ сообщения (фактические позиции роторов).

Виды текстов:

1. Открытый текст, который нужно зашифровать
2. Шифротекст, который нужно расшифровать.

Тестовые наборы:

Некоторое количество заране подготовленных наборов ключей, открытых текстов и ожидаемых шифротекстов.

1.5 Основное содержание работы

1.5.1 Разработка проекта

Общий подход к реализации

Реализация выполняется на языке Haskell с использованием системы сборки Stack.

Вся криптографическая логика (построение подстановки, шифрование и дешифрование символов, шаг роторов) реализуется чистыми функциями без побочных эффектов.

Состояние машины передаётся явно в параметрах функций или с использованием монады State.

Операции ввода-вывода (IO) допускаются только в модулях, отвечающих за взаимодействие с пользователем, файловой системой и логированием.

Все принятые допущения, связанные с недостатком подробной открытой документации машины HX-63, фиксируются в файле docs/design.md.

Архитектура и модульная структура

Проект разбивается на модули с разделёнными зонами ответственности:

1. `Types` — основные типы данных;
2. `Alphabet` работа с внутренним алфавитом $0 \ldots 40$;
3. `Rotor` — представление ротора, его проводки, штифтов и позиции;
4. `Modifier` — входной и re-entry модификаторы;
5. `Stepping` — логика нерегулярного шага роторов;
6. `SignalPath` — построение полной подстановки сигнала (включая re-entry);
7. `Machine` — состояние машины и операции над одним символом;
8. `Key` — внутренний ключ и ключ сообщения;

9. `Config` — загрузка, сохранение и валидация ключей;
10. `CLI` — интерфейс командной строки, логирование и работа с файлами.

Основные типы данных

Определяются следующие ключевые типы (имена могут быть уточнены на этапе проектирования):

1. `Rotor` — ротор (проводка, массив штифтов, текущая позиция);
2. `MachineState` — полное текущее состояние машины;
3. `InternalKey` — долговременный ключ (порядок роторов, проводки, штифты, модификаторы);
4. `MessageKey` — ключ сообщения (начальные позиции роторов);

Модель пути сигнала и механизм re-entry

Реализуется функция построения полной подстановки сигнала в текущем состоянии машины.

В модели используется пространство из 41 позиции ($0 \ldots 40$), из которых $0 \ldots 25$ соответствуют буквам латинского алфавита, а $26 \ldots 40$ — служебным re-entry линиям.

Реализация шага роторов

Реализуется детерминированная функция шага, которая на основании текущего состояния штифтов определяет, какие роторы должны продвинуться.

Конкретный алгоритм шага является принятым допущением и документируется.

Шифрование и дешифрование

Шифрование и дешифрование реализуются как операции над одним символом с последующим обобщением на целые тексты. В каждом состоянии машины с помощью функции buildSubstitution строится подстановка, которую машина выполняет в данный момент (с учётом текущего положения роторов, модификаторов и механизма `re-entry`).

- При шифровании к входному символу применяется эта подстановка, после чего выполняется шаг роторов.
- При дешифровании строится функция, обратная к полученной подстановке, и применяется уже она. После обработки символа выполняется тот же самый шаг роторов, что и при шифровании.

Такой подход позволяет гарантировать корректное восстановление исходного текста при наличии одинаковых ключевых установок и не требует моделирования недокументированных особенностей физического переключения режимов настоящей машины HX-63. На основе операций над одним символом реализуются функции шифрования и дешифрования целых текстов.

Управление ключами и работа с файлами

Программа реализует:

1. генерацию случайного валидного внутреннего ключа;
2. установку ключа сообщения;
3. сохранение и загрузку ключей из файлов в читаемом формате (YAML или JSON);

4. чтение открытых текстов и запись результатов шифрования/дешифрования в файлы;
5. валидацию загружаемых данных.

Интерфейс командной строки и логирование

Реализуется консольный интерфейс с минимальным набором команд (`encrypt`, `decrypt`, `gen-key` и др.).

Все существенные действия пользователя фиксируются в логе.

Обработка ошибок

Реализуется обработка ошибок ввода (некорректные символы, неверный формат файлов ключей, отсутствие файлов и т.д.) с понятными сообщениями для пользователя.

Отчётная документация

Подготавливается отчёт, включающий:

1. титульный лист;
2. содержание;
3. введение (с описанием предметной области и не менее пяти источников);
4. описание разработки проекта и технических особенностей реализации;
5. примеры работы программы;
6. руководство по эксплуатации;
7. заключение (с выгрузкой истории коммитов и диаграммой Ганта);
8. список литературы;
9. приложения.

1.6 Основные требования к выполнению работы

1.6.1 Назначение программного комплекса

Разрабатываемый программный комплекс предназначен для моделирования основных криптографических принципов роторной шифровальной машины НХ-63 в учебных целях. Комплекс позволяет выполнять шифрование и дешифрование текстов, исследовать влияние параметров ключа и демонстрировать работу механизма re-entry.

1.6.2 Состав разрабатываемого программного комплекса

В состав программного комплекса входят:

1. исходный код проекта на языке Haskell, организованный с использованием системы сборки Stack;
2. модули, реализующие модель машины, работу с ключами, интерфейс командной строки, логирование и обработку ошибок;
3. набор автоматизированных тестов на базе библиотеки QuickCheck;
4. файлы конфигурации и примеры ключей;
5. отчётная документация.

1.6.3 Технические характеристики

Программное обеспечение должно обеспечивать следующие характеристики:

1. реализация на языке Haskell (компилятор GHC, сборка через Stack);
2. использование чистых функций для всей криптографической логики (построение подстановки, шифрование/дешифрование символа, шаг роторов);
3. поддержка чтения и записи текстовых файлов и файлов ключей;
4. наличие консольного интерфейса взаимодействия с пользователем;
5. ведение лога действий пользователя;
6. корректная обработка ошибок ввода и данных с информативными сообщениями;
7. возможность генерации, сохранения, загрузки и валидации ключей.

1.6.4 Функциональные требования

Программный комплекс должен обеспечивать выполнение следующих функций:

1. генерация случайного валидного внутреннего ключа;
2. установка ключа сообщения (начальные позиции роторов);
3. сохранение и загрузка ключей из файлов;
4. шифрование текста (из файла или стандартного ввода);
5. дешифрование текста (из файла или стандартного ввода);
6. вывод текущего состояния машины (позиции роторов);
7. логирование основных действий пользователя;
8. обработка некорректного ввода и ошибок работы с файлами;
9. проверка критических свойств модели с помощью библиотеки QuickCheck

1.6.5 Требования к используемым средствам

При разработке математического, программного и информационного обеспечения должны преимущественно использоваться типовые и стандартные средства языка Haskell, а также существующие библиотечные решения. Допускается использование внешних библиотек (в частности, QuickCheck, optparse-applicative, библиотек для работы с YAML/JSON и др.) при условии обоснования целесообразности их применения. Реализация основной криптографической логики выполняется самостоятельно и не должна опираться на готовые криптографические пакеты.

1.7 Этапы и сроки выполнения работы

Работа выполняется в сжатые сроки (до 9 октября 2026 г.). Каждый этап сдаётся отдельно. Разработка ведётся последовательно, с содержательными коммитами. Перечень этапов выполнения работы и сроки перечислены в таблице 1.

№	Этап	Содержание работы	Планируемый срок
1	Выбор предметной области и составление технического задания	Выбор и изучение предметной области, анализ работы шифровальной машины	до 29.09
2	Проектирование	Разработка структуры проекта, диаграммы модулей, основных типов данных, выбор и обоснование библиотек	до 01.10
3	Реализация базовой модели	Типы данных, роторы, модификаторы, функция шага роторов	до 03.10
4	Реализация криптографического ядра	Построение подстановки (`SignalPath` + `re-entry`), `encryptChar` / `decryptChar`, шифрование и дешифрование текстов	до 05.10
5	Работа с ключами и файлами	Генерация, сохранение, загрузка и валидация ключей + чтение/запись текстов	до 06.10
6	Интерфейс, логирование и обработка ошибок	Логирование действий пользователя, обработка ошибок ввода и файлов	до 07.10
7	Тестирование	Написание тестов QuickCheck	до 08.10
8	Разработка документации	Написание отчёта	до 09.10

Таблица 1. *Планируемые сроки выполнения этапов работы*

1.7.1 Диаграмма Ганта

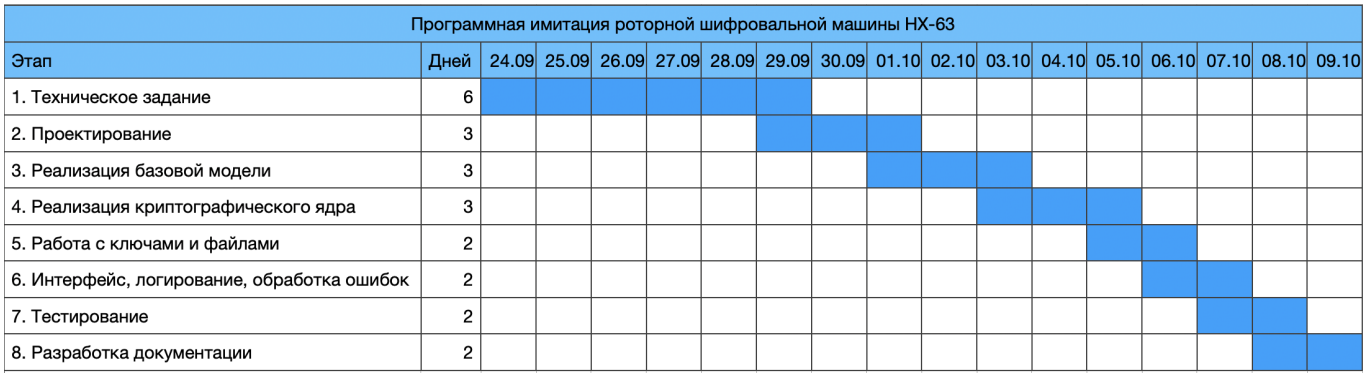


Рис. 1. *Диаграмма Ганта*

1.8 Результаты работы

1. Этап 1. Написание технического задания
   Результатом этапа является согласованное и защищённое техническое задание, а также его распечатанный, сшитый и подписанный у преподавателя экземпляр.
2. Этап 2. Проектирование
   Результатом этапа является описание структуры проекта, диаграмма типов и модулей, описание основных типов данных и перечень используемых библиотек с кратким обоснованием их выбора.
3. Этап 3. Реализация базовой модели
   Результатом этапа является реализация основных типов данных модели, модулей, отвечающих за роторы, модификаторы и шаг роторов, а также соответствующие коммиты в репозитории.
4. Этап 4. Реализация криптографического ядра
   Результатом этапа является реализация функций построения подстановки с учётом механизма `re-entry`, функций шифрования и дешифрования одного символа и целых текстов, а также подтверждение свойства обратимости на примерах.
5. Этап 5. Работа с ключами и файлами
   Результатом этапа является реализация генерации, сохранения, загрузки и валидации ключей, а также чтения открытых текстов и записи результатов шифрования и дешифрования в файлы.
6. Этап 6. Интерфейс, логирование и обработка ошибок
   Результатом этапа является работоспособный консольный интерфейс, реализованное логирование действий пользователя и обработка основных ошибок ввода и работы с файлами.
7. Этап 7. Тестирование
   Результатом этапа является набор тестов на основе библиотеки QuickCheck, проходящий тест обратимости шифрования, а также дополнительные тесты ключевых свойств модели.
8. Этап 8. написание отчёта
   Результатом этапа является полный отчёт в соответствии с требованиями, история коммитов репозитория, соответствующая этапам работы, диаграмма Ганта с фактическими сроками. Итоговым результатом работы является полностью работоспособный Stack-проект на языке Haskell и комплект отчётной документации.

Список литературы

[1] Offline rotor cipher machine — [Электронный ресурс] // CryptoMuseum : сайт. — URL: https://www.cryptomuseum.com/crypto/hagelin/hx63/index.htm (Дата обращения: 26.09.26)

[2] Скандальная история последней роторной шифровальной машины — [Электронный ресурс] // Habr : сайт — URL: https://habr.com/ru/articles/576602/ (Дата обращения: 25.09.26)